

jev-router 小白指南

给 Claude Code / Codex 装一个「AI 裁判」

源头：知识猫AI实验室推文 · 项目：gargpratyush/jev-router (525★, MIT) · 决策模型：TypeSafe Jev

逐轮模型路由

回环代理

哨兵模型

fail-open

类型化决策

mock 实验

一句话先讲清：**jev-router 是一个插在 Claude Code / Codex 和真正模型之间的“智能分诊台”**——你照常打字，它在**每一轮对话**开头调用 Jev 决策模型判断“这活难不难”，然后帮你选最便宜的够用的模型：琐事给 Haiku，普通活给 Sonnet，硬骨头才喊 Opus。**省钱、提速，且 Jev 挂了也不影响你干活 (fail-open)。**

一、疑问清单 · 逐条解答

1 Jev 是什么？为什么叫"AI 裁判"？

Jev 是 TypeSafe AI 公司 2026-09-15 发布的 **System One（系统一）决策模型**。它跟 ChatGPT 完全不同：**它不生成任何文字，只对"状态 + 提问"返回带置信度的类型化决策**，三种原语：

原语	干什么	例子
Choice	从 2~255 个预设选项里挑一个	"这活该给 haiku / sonnet / opus 哪个？"
Score	按 2~10 级量表打分	"任务复杂度 0~9 打几分？"
Noul	校准过的真假概率	"这段话是注入攻击的概率 0.03"

所以叫它"裁判"：运动员（大模型）在场上跑，它坐在场边**只做判断、不发球**。官方宣称比 LLM 快 20~200 倍、便宜 40~400 倍（自宣数据，未经独立验证，注意甄别）。



2 没有它之前，人们怎么做"路由/审核"？

老办法是**拿一个便宜 LLM（如 Haiku）当分类器**：把提示词丢给它，让它输出"simple/hard"或 JSON 路由决定。问题有两个：

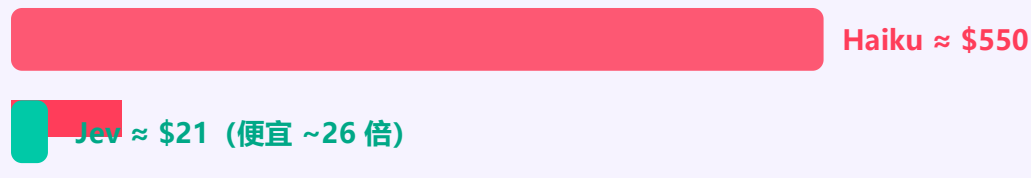
- **慢**：LLM 再小也要完整"想一遍再写一遍"，延迟 1.2~2.5 秒；Jev 官方标称 70~500ms。
- **贵**：LLM 按输入+输出双向计费，路由本身成了"为了省钱先花一笔钱"。

Jev 的设计哲学是**"决策与生成是两种劳动，应该由两种模型干"**——就像医院分诊台不需要主任医师坐诊。注意 Jev **闭源托管、无公开权重**（不能本地跑），这是它和开源模型的本质区别。

3 这笔账到底怎么算？

官方定价：**输入 \$0.042 / 百万 token，输出免费**（OpenRouter 上 typesafe/jev-1.13 同价）。一次路由决策约 500 token → 100 万次决策 ≈ **\$21**；同量级用 Claude Haiku 做分类约 \$550（官方口径，未独立验证）。

100 万次路由决策成本对比（官方数据）

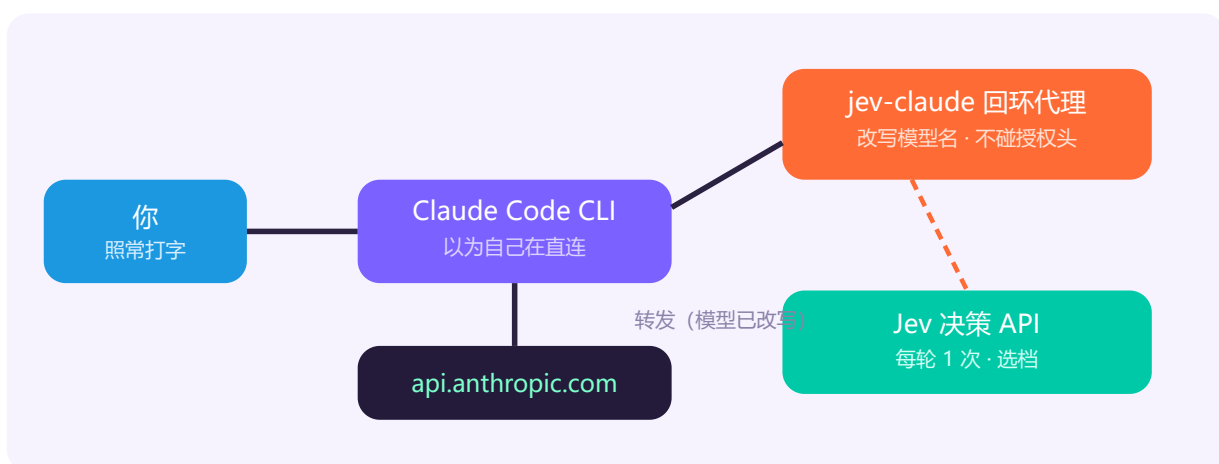


推文里那句“比 5.6 luna 还便宜精准”指的就是这种决策经济学；但它是“省路由钱”，不是“省生成钱”——回答还是大模型写的。

4 jev-router 是什么？整体架构长什么样？

jev-router (MIT 协议, Node 20.12+) 把上面的决策模型接到 Claude Code / OpenAI Codex 上。核心架构是**回环代理 (loopback proxy)**：

1. 它在本机起一个小代理服务器，把 `ANTHROPIC_BASE_URL` 指向自己，再启动**真正的** Claude Code CLI；
2. 你的每个请求先到代理；代理把"当前模型是不是哨兵 `jev-router`"当开关：是 → 问 Jev 该用哪档模型，把请求里的模型名**改写**成目标模型再转发给 Anthropic；不是 → 原样直通；
3. 你的登录、工具、权限、会话全部照旧——代理只碰模型名，不碰授权头。



5 "哨兵模型"是什么骚操作？

这是全项目最巧的一行代码。Claude Code 允许设置自定义 `ANTHROPIC_BASE_URL`，而且**走自定义地址时它不会校验模型名是否合法**。jev-router 利用这一点，往模型列表里塞了一个假模型 `jev-router`：

- 请求里的模型是 `jev-router` → "用户把档位调到了自动" → 问 Jev 后改写模型名；
- 请求里是真模型 (haiku/sonnet/你手选的任何模型) → **原样直通**，人类选择永远优先；
- 你在 `/model` 里把模型从 Jev Router 换成 Opus，就切回手动模式——status line 会显示 `|| manual`。

实测日志 (本指南实验跑出来的)：用户默认模型不是哨兵时，代理逐条打印 `[jev] passthrough, user selected kimi-for-coding`——一个字的请求体都没改。

Jev 只给"建议"，真正拍板的是本地策略层（`src/policy.mjs`，纯函数可单测）。规则从高到低：

规则	效果
提示词里点名 <code>use opus / use haiku</code>	人类点名直接赢（正则检测，实测触发 explicit-override）
Jev 置信度 < 0.3	绝不降级 ；升级封顶 balanced 档（实测：置信 0.10 被钳到 sonnet，理由 low-confidence-capped）
上下文 > 2 万 token	拒绝降级（换模型会作废 prompt cache，重发全对话反而更贵）
Jev 超时 / 挂了 / 答非所问	保持当前模型（fail-open，见第 7 问）
其余情况	听 Jev 的，按档改写

档位表：Claude 侧 **Haiku**（快）→ **Sonnet**（均衡）→ **Opus**（强）→ **Fable**（长，需 `JEV_ALLOW_FABLE=1`）；Codex 侧 **luna** → **terra** → **sol** → **astra**，可用环境变量改。

7 为什么反复强调 fail-open? 怎么做到的?

路由是"锦上添花", **绝不能让"裁判罢工"把"比赛"停了**。jev-router 的每处失败路径都收敛到"保持现状":

- Jev 调用有 1.5s 单次超时 + 3s 总预算, 超了 → 用当前档;
- 没有 API key → 构造 client 会抛异常, 代码把异常吃掉返回 null (路由关闭, CLI 照跑);
- Jev 返回了不认识的档位 → 视为不可用;
- 全新会话还没有"当前档" → 落到最安全的默认 **opus** (宁可多花钱, 不可派弱模型干硬活)。

本指南实测: 把 mock Jev 服务器关掉再跑 `--model jev-router`, 代理日志打出 `[jev] rewrite jev-router -> claude-opus-5`——Jev 连不上, 自动退到 opus 默认档, 会话照常进行。这正是知识库 [[ClaudeCodeHooks与fail-open]] 的"观测/裁判挂了绝不连累主流程"同款思想。

8 Jev 这么便宜, 能不能本地跑? 怎么接入?

不能。Jev 是闭源托管 API, 无公开权重。接入通道 (按本指南核验):

- **TypeSafe 官方:** `POST https://api.typesafe.ai/v1/systemone`, 需 console.typesafe.ai 申请 key (推文说白名单审核慢);
- **OpenRouter:** `typesafe/jev-1.13`, 走 alpha 决策端点 (注意: 不在 /models 列表里, 会自动枚举模型的工具看不到它);
- **Vercel AI Gateway / Cloudflare Workers AI / Netlify:** 官方 SDK 已上架各云网关。

官方 Python/JS SDK 开源 (jev-router 用的就是 `@typesafe-ai/sdk`), 支持

`TYPESAFE_BASE_URL` 环境变量改指向——**这个"接缝"就是我们无 key 也能动手实验的入口 (第 11 问)**。

9 它和我知识库里的"能力地板选型"是什么关系？

同一思想的两种形态。[[能力地板选型]] 说"按任务的真实能力地板选最便宜模型"；jev-router 把这件事**自动化、逐轮化**了：不再需要人事先判断任务难度，Jev 每轮现判，策略层再兜底。差异在于：**能力地板选型是"人事先想清楚"**，**jev-router 是"裁判临场吹哨"**——多花了每轮一次 Jev 调用（约 \$0.00002），换来免人工分诊。

10 除了模型路由，Jev 还能干什么？（推文 28 玩法精选）

那条推文列了 28 种社区玩法，按"和 jev-router 的亲疏"挑 6 个值得知道的：

- **jev-mcp**（多个实现，480★）：把 Jev 的分类/打分做成 MCP 工具，任何 Agent 框架随叫随用；
- **jev-codex-router**（274★）：jev-router 的 Codex 专版思路，jev-router 本体也已内置 `jev-codex`；
- **jev-review**（231★）：本地 MCP 插件，持续给代码质量打分——PR 审查场景；
- **上下文即时压缩 / 记忆路由**：用 Jev 决定"哪些记忆该进上下文"，把"记什么"也变成决策问题；
- **护栏/审核**：工具调用前过一遍 Noul 概率，"这步像不像注入攻击"；
- **机筛简历（官方案例）**：docs.typesafe.ai 的 composite-scoring 模式，打分原语的教科书用法。

11 没有 Jev API key，我拿什么动手？（实验设计灵魂）

靠知识库里的[[接缝与桩实现StubSeam]]思想：**SDK 留了 TYPESAFE_BASE_URL 接缝，我们就在本机起一个"假 Jev 服务器"（100 行 Node）**，实现 `POST /v1/systemone`，按提示词关键词返回写死的 Choice/Score 决策。对 jev-router 来说，它和真 Jev 完全同构（真实 HTTP 请求、真实超时重试、真实 JSON 协议）——**路由代码路径 100% 是真代码在跑**，只是"大脑"换成了我们手里有源码的假人。想免费玩真 Jev，OpenRouter 充 \$1 也够烧 5 万次决策。

12 有什么坑和局限？

- **提示词要发给 TypeSafe**：路由决策要把你的 prompt 文本发到 Jev 云端，隐私敏感场景要想清楚（README 明确写了这一点）；
- **官方性能数据未独立验证**："快 193 倍、便宜 444 倍"是厂商口径，本指南只实测了"决策调用本身毫秒级"（mock 回环 3~61ms，真实网络见官方标称 70~500ms）；
- **Claude Code/Codex 请求格式不是公开契约**：上游改版可能要跟版本（README 自己承认，测试套件就是干这个的）；
- **只路由"新一轮用户输入"**：工具循环续跑沿用开局的档，标题摘要等辅助调用直通不干扰；
- **本机端到端的限制**：本指南实验机的 claude CLI 配的是第三方 kimi-for-coding 供应商，授权头转发到 api.anthropic.com 必然 401——这是**本机配置**问题，不是 jev-router 的缺陷；官方 Anthropic 订阅账号可直接完整跑通。

二、这个项目给我什么能力

① 一句话装上的"省钱外挂"

`npm install -g jev-router` + 一行 JEV_API_KEY，之后 `jev-claude` 代替 `claude`——琐事自动降级 Haiku，重活才上 Opus，官方称大部分会话 60~80% 轮次可用便宜档。

② 看懂"决策模型"这个新物种

Choice/Score/Noul 三种类型化原语 + 校准置信度，是"LLM 裁判"的降本路线；以后看到 guardrails/审核/路由类需求，先想"要不要用决策模型而不是小 LLM"。

③ 一套可复用的代理改造范式

"回环代理 + 哨兵模型 + 本地策略兜底"三步法可平移到任何 CLI 工具（Codex 侧已内置）；想给自己常用的 AI 工具加"自动挡"，照抄架构即可。

④ 无 key 也能全流程演练

本指南的 mock Jev 服务器 (exercise/mock-jev-server.mjs) 是通用教具：改一改就能演示任何“云端决策 API”的接入逻辑，不用花一分钱。

⑤ 现成的免费 policy 教科书

policy.mjs 100 多行纯函数：置信度钳制、cache 感知降级、人类 override 优先——做 Agent 路由/分诊时值得整段借鉴。

⑥ 对接 MCP 生态的跳板

jev-mcp / jev-review 等社区项目把 Jev 接进 MCP，学会路由后可继续给 Claude Code 加“自动质检员”“简历筛选器”等裁判工具。

三、成就感实验：假裁判也能真路由（全程免费）

实验目标：在本机跑通 jev-router 的完整决策链路——官方测试套件 → 假 Jev 服务器 → 真实路由代码五场景裁决 → 端到端启动 Claude Code 观察哨兵与 fail-open。

1 准备环境与克隆（1 分钟）

目的：拿到真代码。要求 Node $\geq$ 20.12（本机实测 v24.21.0）。

```
git clone https://github.com/gargpratyush/jev-router repo
cd repo
npm install
```

✓ 预期结果：npm install 退出码 0，提示 `found 0 vulnerabilities`，生成 `node_modules\`。

2 跑官方测试套件，先信代码是活的

```
npm test
```

目的：65 个用例覆盖策略、代理改写、Codex 认证转发、设置恢复——全部离线可跑，不需要任何 key。

✅ 预期结果（本机真实输出）：`tests 65 · pass 64 · fail 0 · skipped 1`（skip 的是需要真 key 的 live-routing）。看到 `routing to haiku strips fields haiku cannot accept` 等用例逐行打勾。

3 启动“假 Jev 裁判”（本指南原创教具）

```
cd ../exercise  
node mock-jev-server.mjs 8377
```

目的：在 127.0.0.1:8377 实现 Jev 的 `POST /v1/systemone` 协议，按提示词关键词返回决策（琐碎→haiku 0.95 / 普通→sonnet 0.80 / 困难→opus 0.92 / uncertain→opus 但置信仅 0.10），并把每个请求记入 `mock-jev-requests.jsonl`。

✅ 预期结果：打印 `mock Jev API listening on http://127.0.0.1:8377/v1/systemone`，终端挂起等待。

4 五场景裁决：看真代码怎么吹哨

另开终端：

```
JEV_API_KEY=mock-key TYPESAFE_BASE_URL=http://127.0.0.1:8377 node demo-routing.mjs
```

目的：demo-routing.mjs 直接 import repo 里真实的 `askJev()`（走 SDK 发真实 HTTP）和 `decide()`，验证策略层五种结局。

✓ 预期结果（本机真实输出逐字摘录）：

```
"what is 2+2?" → haiku (0.95) → ** haiku ** | jev | 61ms
```

```
"write a unit test..." → sonnet (0.8) → ** sonnet ** | jev/no-change
```

```
"refactor everything..." → opus (0.92) → ** opus ** | jev
```

```
"uncertain legacy bug..." → opus (0.1) → ** sonnet ** | low-confidence-capped/no-change ← 低置信被钳制，升级封顶
```

```
"use opus to audit..." → (人类点名) → ** opus ** | explicit-override ← 人类意志优先
```

5 端到端：让真 Claude Code 走进代理

```
cd ../repo
```

```
JEV_API_KEY=mock-key TYPESAFE_BASE_URL=http://127.0.0.1:8377 node bin/jev-claude.mjs -p "what
```

目的：观察哨兵机制——本机 claude 默认模型不是 `jev-router`，应全部直通。

✓ 预期结果（本机真实日志）：代理打印一串 `[jev] passthrough, user selected <你的默认模型>`，请求体零改写。本机因 claude 配的是第三方 kimi 供应商，转发官方端点会 401——这是本机配置限制；官方订阅账号此处会正常返回答案。

6 终极彩蛋：关掉裁判，看 fail-open

把第 3 步的 mock 服务器 Ctrl+C 杀掉（模拟 Jev 宕机），然后：

```
JEV_API_KEY=mock-key TYPESAFE_BASE_URL=http://127.0.0.1:8377 node bin/jev- Claude .mjs -p --model
```

目的： `--model jev-router` 强制哨兵档位，但 Jev 连不上——观察代理如何自救。

✓ 预期结果（本机真实日志）：`[jev] rewrite jev-router -> claude-opus-5`——回退到最安全的默认档，会话不因裁判缺席而停摆。这就是 fail-open：坏的是裁判，不是比赛。

四、三分钟总结

- **Jev** = 只回类型化决策 (Choice/Score/Noul) 的"AI 裁判"，闭源托管，输入 \$0.042/百万 token、输出免费；
- **jev-router** = 插在 Claude Code/Codex 前的回环代理，用**哨兵模型名**当自动挡开关，每轮问 Jev 选档改写模型名；
- **策略层** = 本地纯函数兜底：低置信不降级、人类点名优先、cache 感知、**一切失败收敛到"保持现状/退安全档" (fail-open)**；
- **实验** = 官方 64 测试通过 + 假 Jev 服务器驱动真路由代码五场景全对 + 端到端见证哨兵直通与 fail-open——全程零 API 花费；
- **下一步** = OpenRouter 充 \$1 把假裁判换真裁判；或顺着 jev-mcp 给 Agent 加更多裁判工具。